

InMind Talent Intelligence Platform (ITIP)

Progress Report — Gen AI Track

Spring 2026

Carl Wakim

March 2026

Abstract

This report presents the progress on the InMind Talent Intelligence Platform (ITIP), a production-oriented multi-agent AI system for talent acquisition and BMW placement intelligence. The platform features a six-container Docker architecture comprising two independent agent systems (LangGraph and Google ADK), a vector-database-backed RAG pipeline, role-based access control, input/output guardrails, a fine-tuned DistilBERT track classifier, and MCP-based interview scheduling tools. The system is fully functional with synthetic data, achieving 96.55% routing accuracy, 88.5% RAGAS faithfulness, and 97% classifier accuracy. Remaining work includes production PDF ingestion and Whisper-based voice interface testing.

Contents

1	Introduction	3
1.1	Objectives	3
2	System Architecture	3
3	Agent System A — LangGraph Supervisor	4
3.1	Supervisor Agent	4
3.2	Specialist Agents	5
3.3	Endpoints	5
4	Agent System B — Google ADK Skills Matcher	5
4.1	Matching Pipeline	6
4.2	ADK Agent Definition	6
5	MCP Server — Interview Scheduling	6
6	RAG Pipeline and Data Ingestion	7
6.1	Embedding Model	7
6.2	Chunking Strategies	7
6.3	Ingestion Modes	7
6.4	Retrieval	7

7	DistilBERT Track Classifier	8
7.1	Training Configuration	8
7.2	Results	8
8	Guardrails	8
8.1	Input Guardrails	8
8.2	Output Guardrails	9
9	Role-Based Access Control and UI	9
9.1	User Roles	9
9.2	Streamlit UI	10
10	Session Management and Infrastructure	10
10.1	Redis Sessions	10
10.2	Structured Logging	10
10.3	Rate Limiting	10
10.4	Docker Compose	10
11	Evaluation Results	10
11.1	Summary	11
11.2	Routing Accuracy	11
11.3	Retrieval Metrics	11
11.4	Configuration Comparison — Top-K Selection	12
11.5	RAGAS Scores	12
12	Remaining Steps	12
13	Conclusion	13

Introduction

The InMind Talent Intelligence Platform (ITIP) is a multi-agent AI system designed to support talent acquisition workflows at inmind.ai, including candidate screening, job search, HR policy lookup, and BMW Group internship placement management.

The platform serves four distinct user roles — job seekers, HR recruiters, InMind staff, and academy instructors — each with tailored access to specialized AI agents. The system combines retrieval-augmented generation (RAG), skill-weighted candidate matching, classification via fine-tuned DistilBERT, and tool-use through the Model Context Protocol (MCP).

Objectives

- Build a multi-agent system using two distinct frameworks (LangGraph and Google ADK)
- Implement RAG over four domain-specific corpora with per-corpus chunking strategies
- Provide role-based access control with input/output guardrails
- Fine-tune a DistilBERT classifier for BMW internship track prediction
- Expose interview scheduling tools via MCP protocol
- Containerize the full stack with Docker Compose
- Evaluate with retrieval metrics, RAGAS, routing accuracy, and classifier metrics

System Architecture

The platform follows a six-container microservices architecture. All inter-service communication occurs via HTTP — no shared Python imports between services.

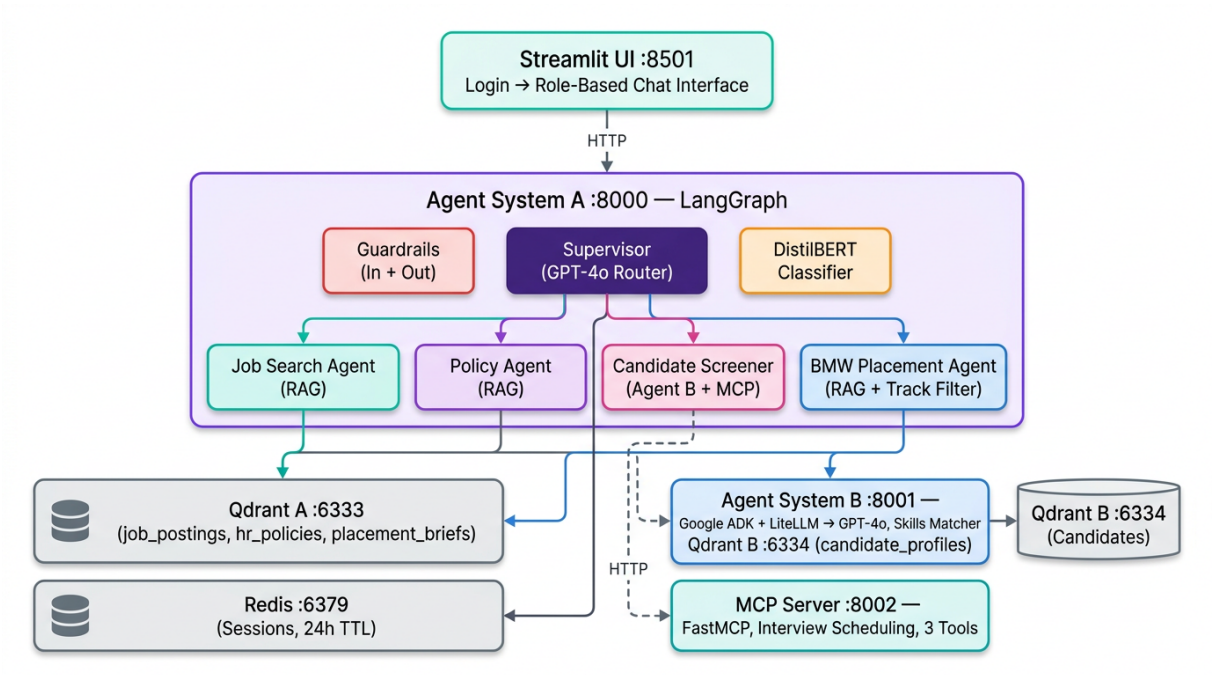


Figure 1: ITIP System Architecture — six-container microservices stack

Table 1: Container summary

Container	Port	Framework	Purpose
Agent System A	8000	FastAPI + LangGraph	Supervisor + 4 specialists, guardrails, DistilBERT, voice
Agent System B	8001	FastAPI + Google ADK	Skills matching with embedding similarity + skill scoring
MCP Server	8002	FastAPI + FastMCP	Interview scheduling tools (mock data)
Qdrant A	6333	Qdrant	Job postings, HR policies, placement briefs
Qdrant B	6334	Qdrant	Candidate profiles
Redis	6379	Redis	Conversation session storage (24h TTL)

Agent System A — LangGraph Supervisor

Agent System A is the primary entry point for all user interactions. It implements a **supervisor-specialist pattern** using LangGraph, where a GPT-4o-based supervisor classifies user intent and routes to one of four specialist agents.

Supervisor Agent

The supervisor receives the user’s message along with conversation history and outputs a JSON routing decision. It operates at `temperature=0.0` for deterministic classification.

The supervisor also enforces role-based access — if the selected specialist is outside the user's allowed list, the request is blocked with an explanatory message.

Available routes: `job_search`, `policy`, `candidate_screener`, `bmw_placement`, and `FINISH`.

Specialist Agents

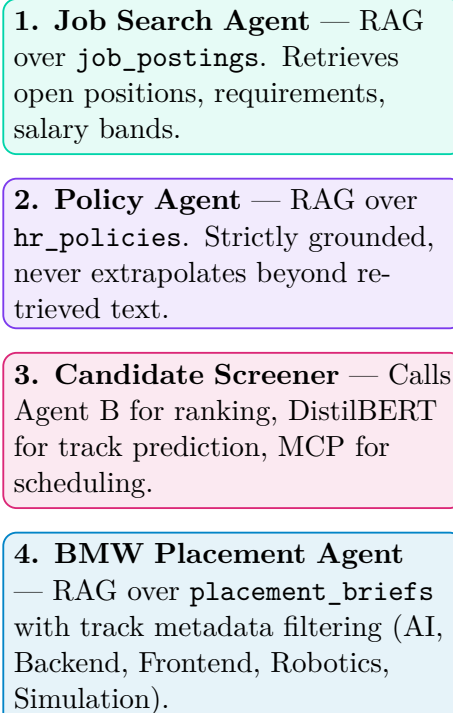
- 
- 1. Job Search Agent** — RAG over `job_postings`. Retrieves open positions, requirements, salary bands.
 - 2. Policy Agent** — RAG over `hr_policies`. Strictly grounded, never extrapolates beyond retrieved text.
 - 3. Candidate Screener** — Calls Agent B for ranking, DistilBERT for track prediction, MCP for scheduling.
 - 4. BMW Placement Agent** — RAG over `placement_briefs` with track metadata filtering (AI, Backend, Frontend, Robotics, Simulation).

Figure 2: The four specialist agents in Agent System A

Endpoints

- `POST /chat` — Main conversational endpoint (synchronous)
- `POST /chat/stream` — Server-Sent Events streaming with real-time node updates
- `POST /chat/voice` — Whisper STT → pipeline → OpenAI TTS (returns MP3)
- `GET /session/{id}` — Retrieve session history (PII-scrubbed)
- `DELETE /session/{id}` — Clear session
- `GET /health` — Health check with Qdrant/Redis connectivity status

Rate limiting is applied via `slowapi`: 30 requests/minute on `/chat`, 5 requests/minute on `/chat/voice`.

Agent System B — Google ADK Skills Matcher

Agent System B is an independent microservice built on the **Google Agent Development Kit (ADK)** with a LiteLLM adapter routing to GPT-4o. It is responsible for skill-weighted candidate ranking.

Matching Pipeline

The matching process follows five sequential steps:

1. **Skill Parsing** — GPT-4o extracts `must_have` and `nice_to_have` skills from the job description
2. **Query Embedding** — The job description is embedded using `text-embedding-3-small` (1536 dimensions)
3. **Semantic Search** — Top-5 candidates retrieved from Qdrant B by cosine similarity
4. **Composite Scoring** — Each candidate receives a composite score:

$$S = 0.4 \times \text{semantic_score} \times 100 + 0.6 \times \text{skill_intersection_score}$$

where skill intersection uses weighted matching (must-have: 3×, nice-to-have: 1×) with word-boundary regex to avoid false positives (e.g., “Java” does not match “JavaScript”)

5. **Summary Generation** — GPT-4o writes a 2–3 sentence recruiter summary per candidate

ADK Agent Definition

The ADK agent registers two `FunctionTools`:

- `adk_parse_skills` — Extracts skills from a job description
- `adk_match_and_rank` — Runs the complete matching pipeline

Shared clients (OpenAI, Qdrant) are injected at startup via a module-level setter function.

MCP Server — Interview Scheduling

The MCP (Model Context Protocol) server exposes three interview scheduling tools via both the MCP protocol and REST endpoints. It uses in-memory mock data with six fictional interviewers.

Table 2: MCP Tools

Tool	Description
<code>find_available_interviewers</code>	Returns available interviewers for a given role, date, and optional BMW track filter
<code>schedule_interview</code>	Books an interview, returns confirmation ID, Zoom link, and status
<code>send_assessment</code>	Dispatches a technical assessment with tracking ID, deadline, and link

The Candidate Screener in Agent A calls these tools via HTTP when it detects scheduling intent keywords in the user message (e.g., “book interview”, “send assessment”).

RAG Pipeline and Data Ingestion

Embedding Model

All collections use `text-embedding-3-small` (1536 dimensions, 8191-token context). The same model is used for both ingestion and query-time embedding.

Chunking Strategies

Each corpus uses a tailored chunking strategy aligned with the project proposal:

Table 3: Per-corpus chunking configuration

Corpus	Strategy	Chunk	Overlap	Rationale
Job Postings	Structure-aware	350 tok	50 tok	Section-specific queries need section-specific chunks
HR Policies	Recursive char split	500 tok	100 tok	Policy prose has natural paragraph boundaries
Placement Briefs	Recursive + metadata	450 tok	90 tok	Mixed doc types; metadata enables pre-filtering
Candidate Profiles	Fixed-size + skill tags	400 tok	80 tok	Semi-structured, dense vocabulary

Ingestion Modes

The ingestion script supports two modes:

- **JSONL mode** (current) — Synthetic data generated by GPT-4o: 21 job postings, 9 HR policies, 14 placement briefs, 33 candidate profiles
- **PDF mode** (production) — Reads from `data/pdfs/` directories with pdfplumber for text extraction, table detection converted to Markdown, and PyMuPDF for image extraction

Point IDs are generated using UUID5 hashing, ensuring re-ingestion is idempotent (no duplicate data).

Retrieval

At query time, the user’s message is embedded and the top- K chunks are retrieved from Qdrant by cosine similarity. The retrieved chunks are formatted with source metadata and passed as context to GPT-4o. The BMW Placement specialist additionally applies a Qdrant metadata filter when a specific track is detected.

DistilBERT Track Classifier

A fine-tuned `distilbert-base-uncased` model classifies candidate text into one of five BMW internship tracks: AI, Backend, Frontend, Robotics, and Simulation.

Training Configuration

Table 4: Classifier training details

Parameter	Value
Base model	<code>distilbert-base-uncased</code>
Training samples	133 (100 BERT snippets + 33 candidate profiles)
Classes	5 (AI, Backend, Frontend, Robotics, Simulation)
Validation	5-fold stratified cross-validation
Epochs	8
Learning rate	2×10^{-5}
Max token length	256
Optimizer	AdamW (weight decay 0.01)
Scheduler	Linear warmup (10% of steps)

Results

Table 5: DistilBERT cross-validation results

Track	Precision	Recall	F1	Support
AI	1.000	0.964	0.982	28
Backend	0.963	0.963	0.963	27
Frontend	0.964	0.964	0.964	28
Robotics	0.929	1.000	0.963	26
Simulation	1.000	0.958	0.979	24
Mean Accuracy	97.01% $\pm$ 2.78%			
Macro F1	97.02%			

The model is loaded lazily on first inference and cached in memory. If model files are not found, the system gracefully falls back without crashing.

Guardrails

Input Guardrails

Input guardrails are applied **before** any agent processing occurs:

Table 6: Input guardrail layers

Guard	Method	Description
Length cap	Hard limit	Messages exceeding 2,000 characters are rejected
Prompt injection	Regex + GPT-4o-mini	Patterns like “ignore previous instructions”; sophisticated attacks caught by GPT-4o-mini classifier
Discriminatory queries	Regex	Detects filtering by protected characteristics (gender, age, race, religion, disability)
PII extraction	Regex	Blocks bulk requests for personal data (emails, addresses, SSNs)
English-only	ASCII heuristic	Rejects messages with <70% ASCII characters

Output Guardrails

Output guardrails are applied **after** the specialist generates a response:

Table 7: Output guardrail layers

Guard	Description
Grounding check	Compares factual tokens in the response against retrieved context. If overlap < 30%, appends a verification warning
Length cap	Truncates responses exceeding 2,500 tokens with a note
Ranking disclaimer	Automatically appends “AI rankings are advisory only” to any response containing ranking keywords
PII scrub (logs)	Redacts emails, phone numbers, SSNs, and credit card numbers before writing to structured log files

Role-Based Access Control and UI

User Roles

Table 8: Role-based specialist access

Role	Allowed Specialists	Demo Credentials
Job Seeker	Job Search	jobseeker / pass123
HR / Recruiter	Candidate Screener, Job Search	hr / pass123
InMind Staff	Policy	staff / pass123
Academy Instructor	BMW Placement, Candidate Screener	instructor / pass123

If the supervisor routes to a specialist outside the user’s allowed list, the system returns an access-denied message. This is enforced server-side in Agent A, not just in the UI.

Streamlit UI

The UI features a dark-themed “Neural Aurora” design with:

- Role-specific accent colors and gradient styling
- Glass-morphism chat bubbles with Material Symbols icons
- Sidebar with real-time Agent A and Agent B health indicators
- Quick-prompt buttons tailored to each role
- Session persistence across page refreshes

Session Management and Infrastructure

Redis Sessions

Conversation history is stored in Redis with a 24-hour TTL. A connection pool is used to avoid creating a new connection per request. Sessions are identified by UUID and can be retrieved (PII-scrubbed) or deleted via REST endpoints.

Structured Logging

All requests are logged to `logs/itip.jsonl` with fields including timestamp, request ID, session ID, routing path, guardrail result, and latency in milliseconds. PII is scrubbed before logging.

Rate Limiting

The `slowapi` library enforces per-IP rate limits: 30 requests/minute on `/chat` and 5 requests/minute on `/chat/voice`.

Docker Compose

All six services are orchestrated with Docker Compose. Health checks ensure proper startup ordering: Qdrant and Redis must be healthy before MCP and Agent B start, and all must be healthy before Agent A starts. Agent A’s Dockerfile installs CPU-only PyTorch to avoid downloading CUDA libraries (~2 GB savings).

Evaluation Results

A comprehensive evaluation was conducted using a 29-question test set spanning six categories.

Summary

Table 9: Overall evaluation results

Metric	Score
Routing Accuracy	96.55% (28/29)
Avg Retrieval Precision @5	23.53%
Avg Retrieval Recall @5	94.12%
Avg MRR	80.59%
RAGAS Faithfulness	88.49%
RAGAS Answer Relevancy	88.10%
RAGAS Context Precision	60.00%
RAGAS Context Recall	90.16%
DistilBERT Accuracy (5-fold CV)	97.01%
DistilBERT Macro F1	97.02%
Guardrail Block Rate	100% (4/4)

Routing Accuracy

The supervisor correctly routed 28 of 29 test queries. The single misroute occurred on a BMW-related job search query that was routed to `bmw_placement` instead of `job_search` — an arguably correct decision given the query’s BMW context.

Table 10: Routing accuracy by category

Category	Correct	Total	Accuracy
Job Search	5	6	83.3%
HR Policy	6	6	100%
Candidate Screening	6	6	100%
BMW Placement	5	5	100%
Guardrail Block	4	4	100%
Out-of-Scope	2	2	100%

Retrieval Metrics

Table 11: Retrieval metrics per collection at $K = 5$

Collection	Avg P@5	Avg R@5	Avg MRR	Questions
job_postings	0.20	0.83	0.53	6
hr_policies	0.20	1.00	1.00	6
placement_briefs	0.32	1.00	0.90	5

High recall across all collections (83–100%) confirms that relevant documents are consistently retrieved. Precision is inherently low at $K = 5$ because most questions have only 1–2 expected documents in 5 retrieved results.

Configuration Comparison — Top-K Selection

Table 12: Impact of retrieval K value on metrics

K Value	Avg P@K	Avg R@K	Avg MRR
$K = 3$	0.3725	0.8824	0.7941
$K = 5$	0.2353	0.9412	0.8059
$K = 7$	0.1765	1.0000	0.8157

$K = 5$ provides the best precision–recall tradeoff. $K = 3$ sacrifices recall for multi-chunk answers; $K = 7$ adds noise without meaningful MRR gain.

RAGAS Scores

Computed over 21 answerable questions using GPT-4o as evaluator:

- **Faithfulness (88.5%)** — 88.5% of answer claims are supported by retrieved context
- **Answer Relevancy (88.1%)** — Answers are highly relevant to the questions asked
- **Context Precision (60.0%)** — 60% of retrieved chunks are useful (room for improvement)
- **Context Recall (90.2%)** — 90% of needed information is present in the retrieved context

Remaining Steps

The following items are implemented but require further testing or production data:

Table 13: Remaining work items

Item	Status	Details
Voice Interface (Whisper + TTS)	Implemented	<code>POST /chat/voice</code> endpoint is coded and functional. Requires OpenAI platform key for Whisper STT and TTS. Needs end-to-end testing with real audio input.
PDF Data Ingestion	Implemented	PDF pipeline with pdfplumber (text), table detection (Markdown conversion), and PyMuPDF (image extraction) is complete. Currently running on synthetic JSONL data; needs real PDF documents in <code>data/pdfs/</code> directories.
Production Data	Pending	Replace synthetic corpora with real job postings, HR policies, placement briefs, and candidate CVs from inmind.ai and BMW Group.
Streaming UI Integration	Partial	<code>/chat/stream</code> SSE endpoint works; Streamlit UI currently uses the synchronous <code>/chat</code> endpoint. Could enhance the UI to show real-time agent activity.
Authentication	Demo only	Current login uses hardcoded credentials. Production deployment would require proper authentication (OAuth, JWT).

Conclusion

The InMind Talent Intelligence Platform is fully functional as a multi-agent AI system with all core components implemented and tested:

- Two independent agent frameworks (LangGraph + Google ADK) communicating via HTTP
- RAG pipeline with per-corpus chunking strategies across four collections
- Role-based access control enforced server-side with four user roles
- Dual-layer input/output guardrails with 100% adversarial blocking rate
- Fine-tuned DistilBERT classifier achieving 97% accuracy on BMW track prediction
- MCP-based interview scheduling with three tool functions
- Full Docker Compose orchestration with health checks and dependency ordering

- Comprehensive evaluation: 96.55% routing accuracy, 88.5% RAGAS faithfulness

The system is ready for demonstration with synthetic data and requires production data ingestion (PDF pipeline) and voice interface testing as final steps.